

Claude Code Insights

0 messages across 0 sessions (1 total) | to

At a Glance

What's working: You've taken the important first step of setting up Claude Code and opting into usage analysis, which shows an intentional approach to adopting AI-assisted development. You're starting with a clean slate — no bad habits to unlearn — which is actually a great position to be in. [Impressive Things You Did](#) →

What's hindering you: On Claude's side, there's nothing to diagnose yet since there are no recorded sessions — so the biggest blocker right now is simply getting started. On your side, the main friction is the cold-start problem: without a CLAUDE.md or any project context established, your first sessions will require more back-and-forth than necessary. [Where Things Go Wrong](#) →

Quick wins to try: Before your first real session, create a CLAUDE.md file in your project root that describes your codebase structure, conventions, and preferences — this will dramatically improve Claude's responses from the start. Then try using your first session as an exploration session where you ask Claude to map your codebase and generate documentation, which both builds context and gives you an immediate useful artifact. [Features to Try](#) →

Ambitious workflows: As models get more capable in the coming months, you'll be able to hand Claude a feature spec and have it autonomously write tests, implement the feature, and iterate until everything passes — no hand-holding required. Start building the habits now (clear specs, good test infrastructure, well-documented projects) so you're ready to take advantage of autonomous multi-step workflows when they land. [On the Horizon](#) →

What You Work On

How You Use CC

Impressive Things

Where Things Go Wrong

Features to Try

New Usage Patterns

On the Horizon

Team Feedback

0

MESSAGES

+0/-0

LINES

0

FILES

0

DAYS

0

MSGs/DAY

WHAT YOU WANTED

No data

TOP TOOLS USED

No data

LANGUAGES

No data

SESSION TYPES

No data

How You Use Claude Code

Based on the data provided, **there are no recorded sessions, messages, or interactions to analyze**. Your usage data shows zero sessions, zero messages, zero hours, and zero commits, with no date range, tools, goals, or outcomes captured. This means either you have not yet started using Claude Code, your usage data was not properly tracked or exported, or the data collection period captured an empty window of activity.

Because there is **no interaction history whatsoever**, it's impossible to characterize your style — whether you tend to iterate quickly with rapid-fire prompts, provide detailed upfront specifications, interrupt Claude mid-task, or let it run autonomously. There are no friction points, no satisfaction signals, no language preferences, and no tool usage patterns to draw from. **To generate a meaningful analysis, at least a few sessions of real usage data would be needed.** If you believe this is an error, it may be worth verifying that your Claude Code client is properly configured to log and export session telemetry.

Key pattern: No interaction style can be determined as the dataset contains zero sessions and zero recorded activity.

USER RESPONSE TIME DISTRIBUTION

No response time data

Median: 0.0s • Average: 0.0s

MULTI-CLAUDING (PARALLEL SESSIONS)

No parallel session usage detected. You typically work with one Claude Code session at a time.

USER MESSAGES BY TIME OF DAY

PT (UTC-8)

No time data

TOOL ERRORS ENCOUNTERED

No tool errors

Impressive Things You Did

Your usage data appears to be empty, so there are no recorded sessions to analyze yet.

Getting Started with Claude Code

You've taken the first step by setting up Claude Code, which positions you well for AI-assisted development. Once you begin using it, your workflow patterns and strengths will become clear.

Clean Slate, Unlimited Potential

You have a fresh environment with no prior friction points or failed sessions to work around. This means you can establish best practices from the very start without any legacy habits to unlearn.

Ready for Data-Driven Insights

You've already opted into usage tracking and analysis, which shows a thoughtful approach to understanding your own productivity. As you start logging sessions, you'll be able to identify your most effective patterns and optimize accordingly.

WHAT HELPED MOST (CLAUDE'S CAPABILITIES)

No data

OUTCOMES

No data

Where Things Go Wrong

There is no usage data available to analyze, so friction points cannot be identified from your Claude Code sessions.

No Recorded Sessions

You have zero recorded sessions in the dataset, which means either you haven't started using Claude Code yet or your usage isn't being captured. Try initiating a few sessions so that meaningful patterns can be identified.

- Zero sessions logged means no workflow patterns can be detected or optimized
- Without session data, it's impossible to determine whether you're encountering tool errors or inefficient prompting habits

Missing Tool Usage Patterns

Your top tools list is empty, suggesting you haven't engaged with any of Claude Code's core capabilities like file editing, search, or bash commands. Start by experimenting with common tools so you can build efficient workflows.

- No tool usage recorded means you may be unaware of capabilities like automated file editing or project-wide search
- Without tool engagement data, there's no way to identify whether you're underutilizing powerful features like batch operations or test running

No Goals or Outcomes Tracked

You have no recorded goals, outcomes, or satisfaction metrics, making it impossible to assess what you're trying to accomplish or how well Claude Code is serving you. Begin by using Claude Code for concrete tasks so that success patterns and pain points can emerge.

- Zero commits suggest you haven't completed any code-producing workflows that could be evaluated for quality or efficiency
- No satisfaction or success data means there's no baseline to measure improvement against as you refine your usage habits

PRIMARY FRICTION TYPES

No data

INFERRED SATISFACTION (MODEL-ESTIMATED)

No data

Existing CC Features to Try

Suggested CLAUDE.md Additions

Just copy this into Claude Code to add it to your CLAUDE.md.

Copy All Checked



Project Overview

Describe your project, tech stack, and key conventions here so Claude has context from the start of every session.

Copy

With no session data captured yet, establishing a CLAUDE.md now means Claude will have project context from your very first real session, avoiding repetitive explanations.



Common Commands

- Build: ``<your build command>``
- Test: ``<your test command>``
- Lint: ``<your lint command>``
- Deploy: ``<your deploy command>``

Copy

Pre-documenting your common commands prevents you from having to tell Claude how to build, test, and lint in every future session.



Coding Conventions

- Preferred language/framework: `<fill in>`
- Always prefer `<X>` over `<Y>` for `<reason>`
- Error handling pattern: `<describe>`

Copy

Capturing coding preferences upfront eliminates the most common source of repeated instructions in future sessions.

Just copy this into Claude Code and it'll set it up for you.

Custom Skills

Reusable prompts that run with a single `/command` for repetitive workflows.

Why for you: Since you're just getting started, setting up a `/commit` or `/review` skill now will save significant time as your usage grows and you establish repeating workflows.

```
mkdir -p .claude/skills/commit && cat  
> .claude/skills/commit/SKILL.md << 'EOF'  
Generate a conventional commit message for  
staged changes.  
1. Run `git diff --cached` to see staged  
changes.  
2. Write a commit message following  
Conventional Commits format (feat:, fix:,  
chore:, etc.).  
3. Ask the user to confirm before  
committing.  
EOF
```

Copy

Hooks

Shell commands that auto-run at specific lifecycle events like after file edits.

Why for you: Hooks let you enforce quality gates (formatting, linting, type-checking) automatically from your very first session, preventing bad habits before they start.

```
# Add to .claude/settings.json:  
{  
  "hooks": {  
    "afterEdit": [  
      {  
        "command": "npx prettier --write  
$CLAUDE_EDITED_FILES",  
        "description": "Auto-format edited  
files with Prettier"  
      }  
    ]  
  }  
}
```

Copy

Headless Mode

Run Claude non-interactively from scripts and CI/CD pipelines.

Why for you: Even before heavy interactive usage, headless mode lets you integrate Claude into your existing workflows for automated lint fixes, code reviews on PRs, or batch operations.

```
claude -p "Review the changes in this diff  
for bugs, security issues, and style  
problems. Be concise." --allowedTools  
"Read,Bash" < <(git diff HEAD~1)
```

Copy

New Ways to Use Claude Code

Just copy this into Claude Code and it'll walk you through it.

Bootstrap Your CLAUDE.md Now

Create a CLAUDE.md before your first real session to give Claude immediate project context.

Your usage data shows zero sessions so far, which means you have a clean slate. The single highest-ROI action is creating a CLAUDE.md with your project overview, tech stack, common commands, and coding conventions. Users who do this report significantly fewer repeated instructions and faster session starts. Even a minimal CLAUDE.md pays dividends immediately.

PASTE INTO CLAUDE CODE:

```
Help me create a CLAUDE.md for this project. Scan the repo to detect the language, framework, build tools, test framework, and linting setup. Then generate a CLAUDE.md with sections for: Project Overview, Tech Stack, Common Commands, Coding Conventions, and Testing Guidelines.
```

Copy

Start With an Exploration Session

Use your first session to let Claude map your codebase and generate documentation.

Before diving into coding tasks, a codebase exploration session helps Claude (and you) build a mental model of the project structure. This is especially valuable because the generated overview can be added to CLAUDE.md, making all future sessions more productive. Ask Claude to identify key modules, entry points, and architectural patterns.

PASTE INTO CLAUDE CODE:

```
Explore this codebase and give me a structured overview. Identify: 1) the main entry points, 2) key modules and their responsibilities, 3) the dependency graph between major components, 4) any patterns or conventions already in use. Format the output so I can paste it into CLAUDE.md.
```

Copy

Establish a Feedback Loop Early

After your first few sessions, review what you repeatedly told Claude and codify it.

The most impactful CLAUDE.md entries come from instructions you find yourself repeating. After 3-5 sessions, review your conversation history for patterns like 'always use X library' or 'run tests before committing.' Adding these to CLAUDE.md eliminates redundancy and makes Claude feel like it truly knows your project. This analysis tool will be more powerful once you have session data to analyze.

PASTE INTO CLAUDE CODE:

```
Look at my recent Claude Code conversations and identify any instructions I gave more than once. List them so I can add them to my CLAUDE.md to avoid repeating myself.
```

Copy

On the Horizon

AI-assisted development is rapidly shifting from interactive pair programming toward fully autonomous, multi-agent workflows that can plan, execute, test, and ship entire features independently.

Autonomous Test-Driven Feature Development

Claude Code can autonomously write a full feature by first generating comprehensive tests from a spec, then iterating on implementation until all tests pass — no human intervention needed. This closes the loop between intent and verified working code, enabling you to ship features while you sleep.

Getting started: Use Claude Code with the `--dangerously-skip-permissions` flag in a sandboxed environment so the agent can freely run tests, edit files, and iterate autonomously.

PASTE INTO CLAUDE CODE:

I want you to build a complete REST API endpoint for user authentication with JWT tokens. Follow this workflow strictly: 1) First, read any existing code structure and understand the project conventions. 2) Write comprehensive unit and integration tests BEFORE writing any implementation – cover happy paths, edge cases, error handling, and security concerns. 3) Run the tests and confirm they all fail. 4) Implement the feature iteratively, running the test suite after each change. 5) Keep iterating until ALL tests pass with zero failures. 6) Refactor the implementation for clarity and performance while ensuring tests still pass. 7) Provide a summary of what was built, test coverage, and any architectural decisions made. Do not ask me any questions – make reasonable decisions and document your assumptions.

Copy

Parallel Multi-Agent Codebase Migration

Multiple Claude Code agents can work in parallel across different modules of a codebase to perform large-scale migrations — such as upgrading frameworks, converting JavaScript to TypeScript, or refactoring API layers — completing in hours what would take a team weeks. Each agent handles an isolated module, runs its own tests, and the results are merged systematically.

Getting started: Spawn multiple Claude Code instances using parallel subprocesses or terminal sessions, each targeting a different directory or module, coordinating via a shared migration plan document.

PASTE INTO CLAUDE CODE:

You are one agent in a parallel migration effort. Your task: migrate all files in the `/src/services/` directory from JavaScript to TypeScript. Follow these steps: 1) List all `.js` files in `/src/services/`. 2) For each file, create a `.ts` version with proper type annotations, interfaces, and

type-safe imports. 3) Update any corresponding test files to work with the TypeScript versions. 4) Run the TypeScript compiler (`npx tsc --noEmit`) after each file conversion to catch type errors immediately. 5) Fix all type errors before moving to the next file. 6) After all files are converted, run the full test suite to ensure nothing is broken. 7) Delete the original `.js` files only after all tests pass. 8) Write a `MIGRATION_LOG.md` summarizing each file converted, types added, and any issues encountered. Work completely autonomously and do not stop until every file is migrated and all tests pass.

Copy

Self-Healing CI Pipeline Debug Agent

Instead of manually debugging broken CI pipelines, Claude Code can autonomously analyze failing builds, trace root causes across code changes, flaky tests, and environment issues, then apply fixes and re-run the pipeline. This transforms CI failures from developer time-sinks into auto-resolved incidents with detailed postmortems.

Getting started: Point Claude Code at your CI logs and give it permission to read test outputs, edit source and config files, and execute test commands locally to reproduce and fix failures.

PASTE INTO CLAUDE CODE:

Our CI pipeline is failing. Act as an autonomous debugging agent and resolve all failures. Follow this process: 1) Read the CI log output from the latest failed run (check `.github/workflows/` for the pipeline config and any recent log files or run `'git log --oneline -10'` to see recent changes). 2) Categorize each failure as: test failure, build error, linting issue, dependency problem, or environment/config issue. 3) For each failure, trace the root cause – check git blame, recent diffs, and dependency changes. 4) Apply fixes for each issue, starting with the most fundamental (build errors first, then test failures). 5) After each fix, run the relevant test suite or build command locally to verify the fix works. 6) If a test is flaky (passes intermittently), stabilize it by adding retries, fixing race conditions, or mocking external dependencies. 7) Run the full CI pipeline locally (or equivalent test/build/lint commands) to confirm everything passes. 8) Create a single commit with all fixes and write a detailed `CI_POSTMORTEM.md` explaining what broke, why, and how it was fixed. Do not stop until all pipeline stages pass cleanly.

Copy

"The ghost town session: someone spun up the analytics engine with absolutely zero data and watched it try to find meaning in the void"

A usage analysis was run against a completely empty dataset — 0 sessions, 0 messages, 0 hours, 0 commits, no date range, no tools, no goals, no friction, no summaries. It's the digital equivalent of opening an empty fridge and still standing there staring.